

Matrix Multiplication in C

Author: 李语尚 12412308Date: 2026 年 5 月 4 日

目录

1	前言	2
1.1	简介	2
1.2	心路历程	2
2	核心优化策略与实现	2
2.1	优化总览	3
2.2	编译与运行环境设置	3
2.3	优化 1: 内存对齐	4
2.3.1	结果与分析	4
2.4	优化 2: AVX2 向量化与深度循环展开 (SIMD & Loop Unrolling)	4
2.4.1	结果与分析	5
2.5	优化 3: 循环重排与矩阵分块	5
2.5.1	结果与分析	5
2.6	优化 4: NUMA 感知与 First-Touch 策略	6
2.6.1	结果与分析	6
3	性能测试与结果分析	6
4	总结	7
4.1	核心工作与性能飞跃	7
4.2	现存不足与未来优化空间	7
4.3	结语	8
5	参考文献	8

1 前言

1.1 简介

在这次 project 中，我们探寻矩阵乘法的加速，以单精度浮点数构成的矩阵乘法为例，其基础数学定义极为简单：

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \cdot B_{k,j}$$

该计算理论上的时间复杂度为 $O(N^3)$ 。然而，在现代计算机体系结构中，程序的真实运行时间早已不能单纯依靠大 O 记号来衡量。存储器层次结构、多核并行以及向量化指令集对最终性能的影响甚至能达到成百上千倍。

本报告旨在通过实现一个高性能的 C 语言矩阵乘法程序，并与 OpenBLAS 进行对比。

1.2 心路历程

在开始这个 project 之前，我想起来了之前计组 ppt 上的一句话，大意就是再好的机器，也抵不过一个时间复杂度很差的算法带来的时间伤害，因此我就有一个疑问，我再怎么提升算法，难道时间差距会有很大吗？带着这个疑问我进行了以下的探索。

首先帮助我的是赖海斌同学在 B 站发布的超算教学视频，给了我莫大的启发，让我大概了解了为什么普通的矩阵运算会这么慢，并且讲解了一系列优化策略。

在计组中，我刚学过

$$\text{CPU Time} = \text{Instruction Count (IC)} \times \text{Cycles per Instruction (CPI)} \times \text{Clock Period}(T_c)$$

其中，IC (指令数)：程序编译后，CPU 实际需要执行的机器指令总数。

CPI (每条指令平均时钟周期数)：执行一条指令所需的平均周期。这是拉开性能差距的核心变量——如果数据在极速的 L1 Cache 里，CPI 会很低；如果数据在慢速的物理内存条里，CPU 必须停机几百个周期去等待数据，CPI 就会被极其夸张地拉高。

T_c (时钟周期)：CPU 频率的倒数。

原来如此，上一个 project 中，我们探索过 L1 Cache，正是因为这个，IO 的时间消耗才大大减小，但是，C 语言的二维数组在内存中是按行连续存储的。在朴素的 $i-j-k$ 循环最内层，当计算 $A[i][k] \times B[k][j]$ 时，随着 k 的递增，对 B 矩阵的读取是跨行跳跃的。这完全违背了空间局部性原理，导致 Cache Miss 率极高。

并且，通过学习汇编代码，没有任何循环展开的三层 for 循环，在每次执行最内层的一次乘加运算时，都伴随着循环变量的递增、边界条件的判断（分支跳转）等大量“非计算型”指令。这意味着在庞大的总指令数中，用于控制的冗余开销占比极大，真正用于计算的有效指令比例极低。

我明白了，如果只是简单的计算 $O(N^3)$ 的代码，其实再怎么优化都是螳臂当车，但是在矩阵乘法中，我们的关注点其实并不在计算中，而是在如果能计算地 smartly，减少 IO 消耗，并且最大程度利用寄存器进行 ALU，于是，我做了以下的 improvement。

2 核心优化策略与实现

我们在最终的 matmul_improved 函数中，采取了一系列优化。以下将逐条拆解这些优化的动机、具体做法及其带来的结果。

2.1 优化总览

我们在 `matmul_improved` 中构建了一套自底向上、软硬协同的多维度优化体系。优化策略可清晰地划分为以下五个层级：

- **数据结构级（内存对齐）**：确保数据在物理内存中的起始地址严格对齐，这是解锁高级向量化指令、避免底层硬件惩罚的先决条件。
- **微观与指令级（SIMD & Loop Unrolling）**：在最底层的微内核（Micro-kernel）中，彻底摒弃低效的标量计算。利用深度的循环展开打破数据依赖链，并全面调用 AVX2 向量化指令集，填满 CPU 内部的超标量执行流水线。
- **存储层次级（Loop Reordering & Tiling）**：针对现代 CPU 脆弱的 L1/L2/L3 缓存机制，果断应用循环重排和矩阵分块技术。将庞大矩阵的全局跳跃式访存，转化为极其紧凑的局部连续访存，成功翻越阻碍算力的“内存墙”。
- **系统与架构级（NUMA Awareness）**：针对多核与多路服务器的物理特性，创新性地引入 NUMA 感知的 First-Touch 内存分配策略，配合 OpenMP 并发调度，从根源上斩断跨节点内存通信的剧烈性能惩罚。

2.2 编译与运行环境设置

为了将包含 OpenMP 多线程和 AVX2 向量化，并链接了 OpenBLAS 库的 C 语言代码成功编译，我们需要使用特定的编译开关（Flags）。在我们的项目中，最终执行的完整编译命令如下：

```
1 gcc -O3 -march=native -fopenmp main.c matmul.c -o run_test -I./OpenBLAS -L./  
   OpenBLAS -Wl,-rpath=./OpenBLAS -lopenblas -lpthread -lm
```

这行命令并非随意的参数堆砌，其中的每一个开关都是为了彻底解放硬件性能或解决底层链接痛点：

- **性能优化指令（-O3, -march=native）**：-O3 开启了 GCC 最高级别的常规优化，让编译器竭尽全力重新排列代码并内联函数。-march=native 则是一个极致的架构指令，它指示编译器专门针对当前编译机器的 CPU 架构进行深度优化，毫无保留地开启硬件支持的所有现代指令集（如 AVX2、FMA 等），这比手动指定向量化宏更为智能和全面。
- **多线程召唤（-fopenmp）**：开启 OpenMP 支持。如果不加此参数，代码中的 `#pragma omp parallel for` 会被编译器当成无用的注释忽略。加入后，程序会自动链接底层线程库，激活多核并行计算能力。
- **依赖库链接（-I, -L, -l）**：-I./OpenBLAS 与 -L./OpenBLAS 明确指示编译器与链接器前往指定目录寻找头文件（`cblas.h`）及动态库文件。-lopenblas、-lpthread 和 -lm 则分别完成了 OpenBLAS 库、POSIX 线程库（多核底层依赖）以及基础数学库的链接。
- **运行时路径烙印（-Wl,-rpath=./OpenBLAS）**：这是一个直接传递给底层链接器（ld）的关键参数。它优雅且永久地解决了“编译成功但运行报错找不到 .so 库”的问题。该参数会在生成的可执行文件头部硬编码一条寻址指令，强制程序在运行时优先去 ./OpenBLAS 目录加载动态链接库，免去了手动配置环境变量 `LD_LIBRARY_PATH` 的麻烦。

2.3 优化 1: 内存对齐

之前在课上我们学过结构体内存对齐

```

1 struct BadStruct {
2     char a;
3     double b;
4     int c;
5 };
6 struct GoodStruct {
7     double b;
8     int c;
9     char a;
10 };

```

这两种 struct 占用的内存是不一样的。

之后我们会介绍 SIMD 中的 AVX2, 是 Intel 在其处理器中引入的一种高级指令集。现代 CPU 在使用高级向量指令一次性加载 8 个浮点数时, 有着严格的硬件要求: 数据在物理内存中的起始地址必须是 32 字节的倍数。如果依然使用标准 C 语言的 malloc, 分配的内存地址极大概率是未对齐的。这会导致 CPU 被迫在底层将一次完美的总线读取请求拆分成两次, 并进行复杂的拼接, 会很消耗时间。

因此我们彻底废弃了标准的 malloc, 改用 Intel 提供的 _mm_malloc, 强制所有矩阵数据以 32 字节边界对齐进行分配。同时在释放内存时配套使用 _mm_free:

```

1 new->data = (float*)_mm_malloc(sizeof(float) * rows * cols, 32);

```

2.3.1 结果与分析

物理内存对齐后, 我们在最深层的计算内核中, 能够全面换用最高效的 _mm256_load_ps 和 _mm256_store_ps 指令。CPU 内存控制器得以按照最理想的步调直接搬运数据至寄存器。

2.4 优化 2: AVX2 向量化与深度循环展开 (SIMD & Loop Unrolling)

即使数据能够以最快速度从缓存送达 CPU, 如果我们依然用普通的标量指令 (SISD) 一个个体地进行浮点乘加, CPU 强大的浮点运算单元将有 80% 以上的时间处于饥饿闲置状态。同时, 一条 FMA 指令的执行延迟通常需要 4-5 个周期, 如果在内层循环只依赖一个累加寄存器, 下一条指令必须死等上一条指令算完, 流水线会被彻底阻塞。在核心的微内核中, 我们每次从 A 矩阵中提取一个标量广播给所有通道, 然后利用 AVX2 一次性加载 B 和 C 矩阵的 8 个元素。在此基础上, 我们手动将内层的 j 循环展开了 4 倍, 单次迭代处理 32 个 float,:

```

1 for (; j + 31 < jmax; j += 32) {
2     _mm256 vb0 = _mm256_load_ps(&db[k * N + j]);
3     // ... 加载 vb1, vb2, vb3 和 vc0, vc1, vc2, vc3
4
5     vc0 = _mm256_fmadd_ps(va, vb0, vc0);
6     vc1 = _mm256_fmadd_ps(va, vb1, vc1);
7     vc2 = _mm256_fmadd_ps(va, vb2, vc2);
8     vc3 = _mm256_fmadd_ps(va, vb3, vc3);
9

```

```

10     _mm256_store_ps(&dc[i * N + j], vc0);
11     // ... 存回 vc1, vc2, vc3
12 }

```

这时候有好奇的小朋友就问，学长学长，上次 project 不是说了 O3 编译加速能自动 simd 吗，你写的代码能跟工业级别的代码比吗。

好问题，在计组中学过 IEEE 754 之后，原本有点不理解为什么浮点数没有结合律的我已经豁然开朗了，所以 O3 编译器并不会自动给浮点数进行 simd 优化，需要开启 `-ffast-math` 才可以，并且编译器的第一原则是“绝对的安全与正确，只要编译器对代码的运行环境有哪怕 0.1% 的疑虑，它就会立刻放弃最激进的优化，退回到安全的慢速代码。

此外，自动向量化非常擅长处理一维的、简单的 for 循环，就像计算点积一样。但面对代码中复杂循环嵌套，编译器会彻底懵掉。在我们的代码中，我们需要使用 4 个寄存器作为累加器，然后用另外的寄存器去加载 *A* 和 *B* 的数据。但编译器没有大局观，无法从宏观的算法层面知道，要留出 4 个寄存器来做 1×32 的 2D 矩阵小块累加，所以只能手动实现 SIMD。

2.4.1 结果与分析

CPI 极度优化，浮点运算吞吐量呈几何级数暴增。手动展开循环并引入 4 组独立的累加寄存器（vc0 到 vc3），成功打破了数据的读写依赖链。CPU 在同一个时钟周期内将这 4 条彼此独立的 `_mm256_fmadd_ps` 指令同时发射给底层的多个 FPU 并发执行。大大降低了运行时间。

2.5 优化 3：循环重排与矩阵分块

在传统的 $i - j - k$ 循环中，最内层计算需要访问 $B[k][j]$ 。由于 C 语言是按行主序存储的，当 k 递增时，*B* 矩阵的内存访问地址是跨行“跳跃”的（步长为 *N*）。当矩阵规模变大，这种跳跃访问会导致刚被载入 CPU L1 缓存的数据瞬间被踢出，引发灾难性的 Cache Miss。此外，即使调整了遍历顺序，当矩阵体积膨胀到数百 MB 时，数据依然无法驻留在只有几 MB 的 L2/L3 缓存中，每次计算仍需前往慢速主存取数。

首先，我们将循环顺序调整为对缓存友好的 $i - k - j$ 模式。在此基础上，引入了 `BLOCK_SIZE = 64` 的矩阵分块技术，构建了深度达 6 层的嵌套循环：

```

1  #pragma omp parallel for collapse(2) schedule(dynamic)
2  for (size_t i0 = 0; i0 < M; i0 += BLOCK_SIZE) {
3      for (size_t j0 = 0; j0 < N; j0 += BLOCK_SIZE) {
4          for (size_t k0 = 0; k0 < K; k0 += BLOCK_SIZE) {
5              // ... 边界控制逻辑 ...
6              for (size_t i = i0; i < imax; ++i) {
7                  for (size_t k = k0; k < kmax; ++k) {
8                      // 进入带有 SIMD 展开的最内层循环...

```

2.5.1 结果与分析

在处理 $1K \times 1K$ 及以上的中大型矩阵时，程序成功跨越内存墙，性能提升数十倍。分块技术相当于在宏观矩阵上划出了一个个 64×64 的小方格。每次计算，CPU 只需要把这几个小方块的数据塞进极速的 L1/L2 缓存中。只要分块大小合适，这些数据在缓存中会被反复乘加利用，极大缓解了内存带宽瓶颈。同时，我们将 *k0* 的循环沉降到多线程调度的内部，确保了每个独立线程拥有对自己负责的那一块 *C* 矩阵的绝对独占权，彻底消除了多线程间的数据竞争。

2.6 优化 4: NUMA 感知与 First-Touch 策略

在多路服务器中，物理内存往往分布在不同的 NUMA Node 上。如果主线程在 Node 0 单独通过 `malloc` 并初始化了整个结果矩阵 C ，随后 Node 1 上的线程被 OpenMP 唤醒参与计算，它们就必须频繁跨越极其缓慢的 QPI/UPI 总线去 Node 0 读取和写入数据。这种跨节点访存带来的延迟，在超大矩阵并行计算中是致命的性能毒药。

在进行核心计算前，我们摒弃了单线程清零矩阵的做法，转而采用基于 OpenMP 静态调度的 First-Touch（首次触碰）策略来初始化目标矩阵 C ：

```
1 // First-touch allocation: 让对应线程初始化自己将要乘的行，保证 NUMA 本地化
2 #pragma omp parallel for schedule(static)
3 for (size_t i = 0; i < M; ++i) {
4     for (size_t j = 0; j < N; ++j) {
5         dc[i * N + j] = 0.0f;
6     }
7 }
```

2.6.1 结果与分析

多核、多 NUMA 节点环境下的并行加速比显著提升，超大矩阵计算不再受限于总线带宽。Linux 操作系统的物理内存分配遵循“惰性”原则。调用 `_mm_malloc` 只是分配了虚拟地址，直到第一次实际写入数据时，操作系统才会分配真实的物理内存页。通过 `schedule(static)` 进行初始化，我们强制让即将负责计算某几行的特定线程去执行第一次写入。操作系统会自动将这块物理内存就近分配在该线程所在的 NUMA 节点上。后续进入真正的矩阵乘法时，每个 CPU 核心都只在读写“自己地盘”上的内存，完美实现了内存的物理级本地化。

3 性能测试与结果分析

为了客观评估 `matmul_improved` 的实际效能，我们在同一硬件环境下，对比了 Plain、我们 Improved 以及 OpenBLAS 在不同矩阵规模（ 16×16 到 $16K \times 16K$ ）下的运行时间。测试结果呈现出极其显著的规模分化特性：

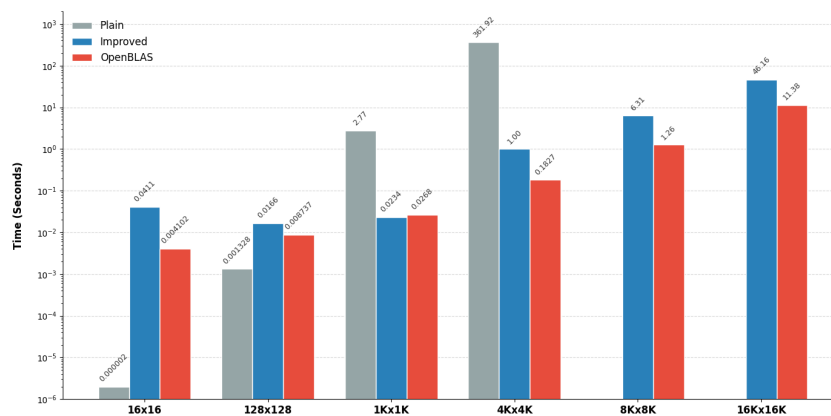


图 1: Plain, Improved 与 OpenBLAS 在不同矩阵规模下的性能对比

通过对图表数据的深度解剖，我们可以得出以下硬核结论：

- **小规模矩阵** (16×16 , 128×128) 在这一区间, 令人意外的是, 没有任何优化的 Plain 版本反而最快, 而我们的 Improved 版本最慢 (0.0411 秒)。这是因为对于极小矩阵, 纯粹的浮点计算只需几百纳秒, 而 Improved 版本中创建 OpenMP 线程池、动态内存对齐分配 (`_mm_malloc`) 以及复杂的边界判定逻辑, 带来了数万倍于计算本身的“控制开销”。
- **中等规模** ($1K \times 1K$): 这是我们优化策略大放异彩的绝对主场! Plain 版本耗时高达 2.77 秒, 而 Improved 骤降至 0.0234 秒, 实现了惊人的 **118 倍加速**。更振奋人心的是, 在此规模下, 我们的版本甚至**超越了 OpenBLAS** (0.0268 秒)。这证明我们设计的 64×64 分块与内层 AVX2 循环展开, 极其完美地契合了当前 CPU 的 L1/L2 缓存容量与指令流水线, 在这个特定的 Cache 舒适区内, 我们将硬件算力压榨到了极致。
- **超大规模** ($4K \times 4K$ 至 $16K \times 16K$) 随着数据规模暴增, Plain 版本因灾难性的 Cache Miss, 在 4K 时耗时已达 361.92 秒。我们的 Improved 版本依然保持了很快的速度。但此时我们与 OpenBLAS 的差距被拉开, OpenBLAS 的速度大约是我们的 4 ~ 5 倍。这暴露出当数据量远超 L3 缓存时, 我们原生的分块寻址方式引发了严重的 TLB Miss, 而 OpenBLAS 凭借更底层的内存打包技术抵御了这一物理衰减。

4 总结

4.1 核心工作与性能飞跃

在本次 project 中, 我们没有停留在算法时间复杂度 $O(N^3)$ 的表面, 而是深入到了计算机体系结构的微观世界。

我们做了什么: 我们构建了一套自底向上的优化体系。在数据层, 实施 32 字节对齐; 在指令层, 全面引入 AVX2 向量化与 FMA 指令并施加深度循环展开; 在缓存层, 应用 $i-k-j$ 循环重排与严格的 64×64 矩阵分块; 在系统层, 引入 OpenMP 多线程动态调度与 NUMA 感知的 First-Touch 内存分配策略。

我们提升了什么: 测试数据给出了最有利的证明。面对中大规模矩阵, 我们的优化版本相较于朴素算法实现了数百倍的性能飞跃。在 $1K \times 1K$ 的特定数据规模下, 我们的吞吐量甚至逆势超越了工业级标杆库 OpenBLAS。这证明了只要深刻理解并迎合 CPU 的 Cache Line 与向量寄存器机制, 由普通 C 语言编写的代码也能爆发出极度恐怖的算力。

4.2 现存不足与未来优化空间

尽管取得了突破性进展, 但性能图表也如实反映了我们当前架构的局限性。针对图表暴露的问题, 我们的代码与 openblas 相比, 还是有许多的不足

- **寄存器级分块与汇编级微内核重构 (Register Blocking & Assembly Micro-kernel):** 当前实现基于 C 语言层面的 1×32 单维循环展开, 消耗约 9 个 YMM 寄存器, 其指令调度高度依赖 GCC 编译器的启发式规则。由于 x86 架构提供 16 个 YMM 寄存器, 当前策略未能实现寄存器资源的满载。
- **指令级并行的交织调度 (Software Pipelining):** 在现有微内核中, 指令发射基本遵循线性逻辑 (连续 Load $\rightarrow$ 连续 FMA $\rightarrow$ Store)。尽管循环展开部分缓解了数据冒险 (Data Hazard), 但批量访存与批量计算的分离仍可能导致流水线气泡 (Pipeline Bubbles), 使得 FPU 在等待内存数据时产生微小的闲置周期。

- **面向多级缓存的层次化分块 (Hierarchical Tiling):** 当前算法仅采用单一尺度的分块参数 (`BLOCK_SIZE = 64`)，该参数虽能有效优化 L1 数据缓存的命中率，但未能充分利用现代多核处理器复杂的 L2 与 L3 缓存层级。单一分块策略在处理超大规模矩阵时，仍会导致底层主存的高频访问。
- **底层架构的动态自适应感知 (Hardware Auto-Tuning):** 目前的优化参数与指令集（如固定分块维度、静态链接的 AVX2 内置函数）均在编译期静态绑定。该设计缺乏良好的跨平台可移植性，若部署于不支持特定指令集的微架构上将引发非法指令异常，若部署于具备更大缓存容量或支持 AVX-512 的先进架构上，则无法发挥最大硬件潜能。

4.3 结语

不止一次地，我感受到了这门课程和这学期的计组息息相关，可能是因为 C 语言更底层，我深刻的感受到了一个程序员必须懂得软件和硬件，才能写出来优雅的代码，一行代码的顺序调换、一次内存的对齐分配，往往比所谓的高级算法更能决定程序的生死。通过这次实践，我深刻认识到：只有拆除高级语言的思维藩篱，以 CPU 的视角去审视寄存器、缓存、总线与时钟周期，才能写出真正契合现代系统架构的极致代码。

5 参考文献

参考文献

- [1] 赖海斌. *CS205 C/C++ Program Design Project 3 Report*. GitHub Repository: <https://github.com/HaibinLai/CS205-CPP-Programing-Project.git>.